

```

# gcov gcov_example.c
File 'gcov_example.c'
Lines executed:87.50% of 8
Creating 'gcov_example.c.gcov'

# cat gcov_example.c.gcov
+.:      0:Source:gcov_example.c
+.:      0:Graph:gcov_example.gcn
+.:      0:Data:gcov_example.gcd
+.:      0:Runs:1
+.:      0:Programs:1
+.:      1:/* Program name : gcov_example.c */
+.:      2:
+.:      3:#include<stdio.h>
+.:      4:
+.:      5:int main()
+.:      6:{
+.:      7:    int i;
+.:      8:    for(i=0; i<10; i++)
+.:      9:    {
+.:     10:      if(i==5)
+.:     11:      {
+.:     12:        continue;
+.:     13:      }
+.:     14:      if(i==11)
+.:     15:      {
+.:     16:        break;
+.:     17:      }
+.:     18:      printf("%d\n", i);
+.:     19:    }
+.:     20:    return 0;
+.:     21:}
    
```

Figure 1: Output of Gcov

the `gdb` command. The transcript below shows the GDB session, which debugs the program `gdb_example.c`.

```

(gdb) file ./a.out
Reading symbols from /root/Desktop/gdb/a.out...done.
(gdb) run
Starting program: /root/Desktop/gdb/a.out
Program received signal SIGFPE, Arithmetic exception.
0x00048422 in main () at gdb_example.c:8
8      div=a/b;
    
```

The message clearly tells us that the run time error is due to division by zero. The command `file` is used to open an executable file for processing by GDB. The `run` command runs the program completely if there are no errors and stops processing with a diagnostic message at the first instance of a run time error. A very rich command set is available for fine tuned debugging of buggy programs and this makes GDB a tool of great importance.

Code coverage tools

Source code coverage analysis tools tell the programmer the number of times each instruction in a program is executed. This information can be further used by the programmer to remove dead code and optimise the frequently executed instructions in a program. GCC offers a very simple and elegant tool for code coverage analysis called Gcov. The program should be compiled as follows to enable the proper working of Gcov: `gcc -Wall -fprofile-arcs -ftest-coverage program_name.c`. The

option `-Wall` shows all the warnings, the option `-fprofile-arcs` is to deal with the execution of branch statements and the option `-ftest-coverage` adds the number of times each line is executed. After compilation, the program should be executed once before calling the Gcov utility. The following command calls the Gcov utility: `gcov program_name.c`. This will create a file called `program_name.c.gcov` which contains the output of Gcov. This file contains the program annotated with the line number and the number of times each line is being executed. Dead code that never gets executed is marked with the symbol `#####`. Figure 1 shows the output of Gcov for a program called `gcov_example.c`.

Static code checking tools

Static code checkers uncover security vulnerabilities and coding mistakes. Static code checking is carried out by analysing the source code or the object code. In static code checking the executable code is never analysed. The standard static code checking utility of Linux was Lint. But nowadays, the most popular static code checker for the Linux environment is Splint (Secure Programming Lint). Splint can uncover unused declarations, the use of variables before definition, type inconsistencies, unreachable code, ignored return values, infinite loops, etc. Consider the program `buggy_program1.c` for a demonstration of Splint's ability to uncover potential errors.

```

#include<stdio.h>
int main()
{
    int i;
    printf("\n%d\n",i);
    i='A';
    printf("%d",i);
    i++;i++;
    printf("\n%d\n",i);
}
    
```

The program can be processed with Splint by using the following command, `splint program_name.c`. This will list many unwanted warnings about potential threats which do not hinder program processing. These warnings can be suppressed by turning on the flag `weak`, as follows, `splint -weak program_name.c`.

The three potential logical errors in the program `buggy_program1.c` are uncovered by Splint after analysing the source code. The three potential problems are the use of variable `'i'` before initialisation, the assignment of a character value to the integer variable `'i'` and the use of the undefined expression `++i++i++`. This shows us that Splint can uncover many potential threats unidentified by the compiler.

Dynamic code checking tools

Dynamic code checking involves the analysis of the executable file and system execution of a program to identify potential threats. Let's discuss a dynamic testing tool called Valgrind, which is used for memory debugging and memory leak detection. Sometimes